Derivatives of the generalized chi-square CDF with respect to its parameters

1 Gradient and Hessian derivations

The generalized chi-square distribution is the distribution of a quadratic function of a normal random vector, or equivalently of a weighted sum of independent chi-square variables. It appears wherever a squared or quadratic quantity is formed from correlated Gaussian noise, most notably as the decision variable of a Bayes-optimal classifier between two normal populations. This appendix derives the gradient and the Hessian — the vector of first derivatives and the matrix of second derivatives — of the generalized chi-square cumulative distribution function (cdf) with respect to the parameters that specify it. Throughout we follow the notation of the generalized chi-square paper.[2]

A generalized chi-square variable, which we write $\tilde{\chi}$, can be described in two equivalent ways, and each way carries its own natural set of parameters. We will need the derivatives of its cdf F with respect to both sets, so we introduce them here.

The canonical form, and the native parameters. In its canonical form, a generalized chi-square variable is a weighted sum of independent non-central chi-square variables, plus an independent normal term:

$$\tilde{\chi}_{\boldsymbol{w},\boldsymbol{k},\boldsymbol{\lambda},s,m} = \sum_i w_i \, \chi'^2_{k_i,\lambda_i} + s\, z + m.$$

Here each $\chi'^2_{k_i,\lambda_i}$ is a non-central chi-square variable with k_i degrees of freedom (the number of squared standard normals summed) and non-centrality λ_i (the summed squared means of those normals); w_i is the weight applied to it; z is a standard normal; s scales that normal term; and m is a constant offset. We call $(\boldsymbol{w}, \boldsymbol{k}, \boldsymbol{\lambda}, s, m)$ the *native parameters*, since they are intrinsic to the distribution and do not refer to any underlying vector.

The quadratic-form, and the boundary parameters. Equivalently, $\tilde{\chi}$ can be written as a quadratic function of a normal vector $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$,

$$\tilde{\chi} = q(\boldsymbol{x}) = \boldsymbol{x}'\mathbf{Q}_2\,\boldsymbol{x} + \boldsymbol{q}_1'\boldsymbol{x} + q_0,$$

with a symmetric matrix $\mathbf{Q}_2$, a vector $\boldsymbol{q}_1$, and a scalar q_0. Because the triple $(\mathbf{Q}_2, \boldsymbol{q}_1, q_0)$ is exactly the coefficient set of a quadratic classification boundary, we call these the *boundary parameters*.

Diagonalizing this quadratic form recovers the canonical form above: the eigenvalues of the matrix $\boldsymbol{\Sigma}^{1/2}\mathbf{Q}_2\boldsymbol{\Sigma}^{1/2}$ are the weights $\boldsymbol{w}$, and the multiplicity of each eigenvalue is the corresponding degree of freedom k_i. This is the conversion between the two representations derived by Das and Geisler.[1]

We will derive the gradient and Hessian of F with respect to each set of parameters.

The Gil-Pelaez inversion and a master formula

Both parameter sets rest on the same piece of machinery, which we set up once here. However $\tilde{\chi}$ is parametrized, it has a characteristic function $\phi(t) = \mathrm{E}\left[e^{it\tilde{\chi}}\right]$, the Fourier transform of its density, and we recover the cdf and the probability density function (pdf) from ϕ by the Gil-Pelaez inversion formula, in the numerically convenient form given by Imhof:[2]

$$F(x) = \frac{1}{2} - \frac{1}{\pi}\int_0^\infty \frac{\mathrm{Im}\left[\phi(t)\, e^{-itx}\right]}{t}\, dt, \qquad f(x) = \frac{1}{\pi}\int_0^\infty \mathrm{Re}\left[\phi(t)\, e^{-itx}\right]\, dt.$$

The key observation is that both of these are *linear* in ϕ: the characteristic function enters only through the bracketed product, while the kernel e^{-itx} and the weight $1/t$ depend on the argument x and the integration variable t, but on none of the distribution's parameters. It will be convenient to give the cdf integral its own name as an operator acting on a function ψ, dropping the constant $\frac{1}{2}$ since no parameter depends on it:

$$T[\psi](x) := -\frac{1}{\pi}\int_0^\infty \frac{\mathrm{Im}\left[\psi(t)\, e^{-itx}\right]}{t}\, dt, \qquad \text{so that } T[\phi] = F - \tfrac{1}{2}.$$

Since every parameter enters F only through ϕ, differentiating F in a parameter θ amounts to differentiating ϕ inside the integral. Writing the derivative of ϕ through its logarithm, $\partial_\theta \phi = (\partial_\theta \log \phi)\, \phi$, and using the linearity of T, we obtain the master formula used throughout this appendix:

$$\boxed{\partial_\theta F = T\left[(\partial_\theta \log \phi)\, \phi\right].}$$

Each derivative then costs only one line of algebra to find $\partial_\theta \log \phi$, after which we are left with an integrand to recognize. Two simple rules do the recognizing every time.

Rule R1: a factor $(1 - 2iw_j t)^{-1}$ raises a degree of freedom by two. The native characteristic function (§1.1) contains the factor $(1 - 2iw_j t)^{-k_j/2}$ for the j-th component. Multiplying the integrand by one more power $(1 - 2iw_j t)^{-1}$ therefore turns it into $(1 - 2iw_j t)^{-(k_j+2)/2}$ and changes nothing else, which is simply the same distribution with its j-th degree of freedom raised by two. Writing $[k_j + n]$ for the distribution with k_j replaced by $k_j + n$,

$$(1 - 2iw_j t)^{-1}\, \phi \;=\; \phi_{[k_j+2]}.$$

Because T is linear, this shift passes straight through to the cdf and pdf, so that $T[\phi_{[k_j+2]}] = F_{[k_j+2]} - \frac{1}{2}$, and similarly for the density. Whenever such a factor appears, we may simply read it as "evaluate at a raised degree of freedom," with no new integral to perform.

Rule R2: a factor it is an x-derivative. The argument x enters the integrand only through the kernel e^{-itx}, and differentiating that kernel in x brings down a factor $-it$. Hence

$$T[\,(it)\,\psi\,] = -\,\partial_x T[\psi],$$

so that $T[(it)\phi] = -f$, and more generally $T[(it)^n\phi] = (-\partial_x)^n F$. The same rule is what lets us compute the derivatives of the density in its argument without any finite differencing: inserting a factor t^n into the inversion integral for f returns $f^{(n)}$ exactly. For instance $f'(x) = \frac{1}{\pi}\int_0^\infty t\ \mathrm{Im}[\phi\, e^{-itx}]\, dt$, an integral of the same kind as the one for f. We use this repeatedly, first for $\partial_s F$ and then throughout the Hessian.

Between them, these two rules turn every derivative below into one of three kinds of object:

1. a cdf or pdf at shifted degrees of freedom, by rule R1;
2. a derivative of the density in its argument, f', f'', . . ., obtained from the same inversion integral by rule R2;
3. or, only when we differentiate in a degree of freedom k_j, a single convergent integral carrying a logarithmic weight (§1.1).

Every one of these is exact — we never finite-difference, and never regularize. The one object that needs care is the second kind: when the normal term vanishes ($s = 0$) and the total degrees of freedom are few, the inversion integral for the density derivatives loses its convergence, and §1.5 computes them there from an exact series instead.

Why we keep two separate parameter sets

The native-parameter derivatives (§1.1–§1.2) and the boundary-parameter derivatives (§1.3–§1.4) answer different questions, and neither is obtained from the other.

The native-parameter derivatives are the general-purpose ones, needed whenever a generalized chi-square variable is the object of interest in its own right, as in signal detection or the study of quadratic forms in Gaussian variables. They also serve as a chain-rule hub for any other parametrization: if some parameter θ maps into $(\boldsymbol{w}, \boldsymbol{k}, \boldsymbol{\lambda}, s, m)$, its derivative follows by combining the native derivatives with the chain rule, as illustrated at the end of §1.1.

The boundary-parameter derivatives serve the quadratic-classification problem, such as computing the sensitivity of a classification error to the boundary coefficients.[3] One might hope to reach them indirectly, by chaining the native derivatives of §1.1 through the quadratic-to-native conversion of Das and Geisler.[1] That route, however, is fragile. The conversion passes through the eigendecomposition of $\boldsymbol{\Sigma}^{1/2}\mathbf{Q}_2\boldsymbol{\Sigma}^{1/2}$, whose eigenvalue multiplicities are the degrees of freedom $\boldsymbol{k}$. These multiplicities are integers, so they are not differentiable functions of the coefficients; moreover the eigenvectors turn singular wherever two eigenvalues coincide. A chain rule built on this map is not stable.

We avoid the difficulty by differentiating the characteristic function of $q(\boldsymbol{x})$ *directly* in the coefficients (§1.3). That route never introduces $\boldsymbol{k}$ as a variable at all: it differentiates the ordinary matrix inverse $\mathbf{M}^{-1}(t) = (\boldsymbol{\Sigma}^{-1} - 2it\,\mathbf{Q}_2)^{-1}$ through the identity $d(\mathbf{M}^{-1}) = 2it\,\mathbf{M}^{-1}(d\mathbf{Q}_2)\mathbf{M}^{-1}$, which is smooth in $\mathbf{Q}_2$ and tracks no individual eigenvalue or eigenvector, so no integer multiplicity ever appears to spoil differentiability. The eigendecomposition returns later, but only as a device for evaluating the resulting integral quickly (§1.3), and even there only through the eigenvalues, never the multiplicities.

So we have two derivative routines for two genuinely different sets of variables, not one computation written twice. What they share is the layer beneath them: the same x-weighted inversion integrands (for f'

and for $\partial_{k_j} F$) and the same two rules. The natural design is therefore a single common integrand at the core, with the two routines built on top of it.

1.1 Gradient with respect to the native parameters

In the canonical form, the characteristic function is

$$\phi(t) = \frac{\exp\!\left(imt - \tfrac{1}{2}s^2t^2 + \sum_j \dfrac{i\lambda_j w_j t}{1 - 2iw_j t} \right)}{\prod_j (1 - 2iw_j t)^{k_j/2}}.$$

Its logarithm is a plain sum of terms,

$$\log\phi(t) = imt - \tfrac{1}{2}s^2t^2 + \sum_j \frac{i\lambda_j w_j t}{1 - 2iw_j t} - \sum_j \frac{k_j}{2}\log(1 - 2iw_j t),$$

so each parameter derivative is straightforward. We take the parameters one at a time: differentiate $\log\phi$, substitute into the master formula $\partial_\theta F = T[(\partial_\theta \log\phi)\,\phi]$, and read off the result with rules R1 and R2.

Offset m. The offset appears only in the single term imt, so $\partial_m \log\phi = it$, and by rule R2 this factor it is one x-derivative:

$$\boxed{\partial_m F = T[(it)\phi] = -\partial_x F = -\,f(x).}$$

Normal scale s. Only the term $-\frac{1}{2}s^2t^2$ depends on s. Using $(it)^2 = -t^2$, its derivative is $\partial_s \log\phi = -s\,t^2 = s\,(it)^2$, and two factors of it are two derivatives in the argument:

$$\boxed{\partial_s F = s\,T[(it)^2\phi] = s\,\partial_x^2 F = s\,f'(x).}$$

Here f' comes from the x-weighted inversion of rule R2, not from finite differencing.

Non-centralities λ_j. Only the exponent's sum involves λ_j, giving

$$\partial_{\lambda_j}\log\phi = \frac{iw_j t}{1 - 2iw_j t}.$$

To recognize this, substitute $u = 2iw_j t$ (so $iw_j t = u/2$) and split off a constant:

$$\frac{iw_j t}{1 - 2iw_j t} = \frac{u/2}{1-u} = \frac{1}{2}\left(\frac{1}{1-u} - 1\right) = \frac{1}{2}\Big[(1 - 2iw_j t)^{-1} - 1\Big].$$

The first piece is a factor $(1 - 2iw_j t)^{-1}$, which by rule R1 raises k_j to $k_j + 2$; the second is ϕ itself. Hence

$$\boxed{\partial_{\lambda_j} F = \tfrac{1}{2}\big(F_{[k_j+2]} - F\big).}$$

Weights w_j. The weight w_j appears both in the exponent and in the power of the denominator, so its derivative has two contributions. Using $\dfrac{d}{dw_j}\dfrac{w_j}{1-2iw_jt} = \dfrac{1}{(1-2iw_jt)^2}$,

$$\partial_{w_j}\log\phi = \underbrace{\frac{i\lambda_j t}{(1-2iw_jt)^2}}_{\text{from the exponent}} + \underbrace{\frac{ik_j t}{1-2iw_jt}}_{\text{from the power}} = k_j\,(it)\,(1-2iw_jt)^{-1} + \lambda_j\,(it)\,(1-2iw_jt)^{-2}.$$

Each term is a factor it times one or two factors of $(1-2iw_jt)^{-1}$. Rule R1 (applied once, then twice) reads off the degree-of-freedom shifts, and rule R2 turns each it into $-\partial_x$:

$$\partial_{w_j}F = k_j\,(-\partial_x)F_{[k_j+2]} + \lambda_j\,(-\partial_x)F_{[k_j+4]},$$

that is,

$$\boxed{\partial_{w_j}F = -k_j\,f_{[k_j+2]}(x) - \lambda_j\,f_{[k_j+4]}(x).}$$

As a check, setting $\lambda_j = 0$ leaves $-k_j f_{[k_j+2]}$, the weighted-sum analogue of the elementary identity $x\,f_{\chi^2_k}(x) = k\,f_{\chi^2_{k+2}}(x)$.

Degrees of freedom k_j. Here the derivative is a logarithm,

$$\partial_{k_j}\log\phi = -\tfrac{1}{2}\log(1-2iw_jt) \implies \boxed{\partial_{k_j}F = \frac{1}{2\pi}\int_0^\infty \frac{\operatorname{Im}\big[\log(1-2iw_jt)\,\phi\,e^{-itx}\big]}{t}\,dt.}$$

This is the one derivative with no finite shifted-dof form, because a logarithm cannot be absorbed by rule R1, so it remains an integral. It nonetheless converges without difficulty: $\log(1-2iw_jt)$ grows only like $\log t$, which the weight $1/t$ easily controls, and it is evaluated by the same inversion machinery as F itself.

All five derivatives converge at first order. The only two that pull down positive powers of $t-\partial_m$ (a factor it) and ∂_s (a factor t^2) — are precisely the ones that are x-derivatives of the integrable density, $-f$ and $s\,f'$. A genuine loss of convergence appears only at *second* order in the location and scale directions, and we defer that discussion to §1.5.

Chaining to any other parametrization. If a downstream parameter θ maps into $(\boldsymbol{w}, \boldsymbol{k}, \boldsymbol{\lambda}, s, m)$, its derivative assembles from the blocks above by the chain rule, $\partial_\theta F = \sum_j (\partial_\theta w_j)\,\partial_{w_j}F + \dots$. The single case where we do not recommend this — the quadratic boundary coefficients — is treated directly, and more stably, in §1.3.

1.2 Hessian with respect to the native parameters

The Hessian collects the second derivatives $H_{ab} = \partial_a\partial_b F$ over all pairs of parameters $a, b \in \{w_j, k_j, \lambda_j, s, m\}$. It is a symmetric matrix, of size $(3n+2)\times(3n+2)$ for a distribution with n components. As with the gradient, the two rules reduce nearly every entry to shifted-dof cdfs and their x-

derivatives; the only entries that remain as integrals are those that differentiate a degree of freedom, and these are the same convergent log-weighted integrals met in §1.1.

The second-order master formula. Differentiating the gradient $\partial_b F = T[L_b\,\phi]$ once more, and writing $L_a \equiv \partial_a \log\phi$, the product rule (with $\partial_a\phi = L_a\phi$) gives

$$\boxed{\partial_a\partial_b F = T\big[(L_{ab} + L_a L_b)\,\phi\big], \qquad L_{ab} \equiv \partial_a\partial_b \log\phi.}$$

Everything new is contained in the second derivatives L_{ab} of $\log\phi$; the products L_aL_b are built from the first derivatives already in hand. Rules R1 and R2 then turn each resulting symbol into a computable quantity.

The derivatives of $\log\phi$. It helps to abbreviate the three recurring pieces: $p \equiv it$, $g_j \equiv (1-2iw_jt)^{-1}$, and $\ell_j \equiv -\tfrac12\log(1-2iw_jt)$. Note $p^2 = -t^2$. Under the two rules, a factor p^a acts as $(-\partial_x)^a$, a factor g_j^n raises k_j by $2n$, and a factor ℓ_j carries the logarithmic weight of a k_j-derivative. In these symbols the first derivatives of §1.1 read

$$L_m = p, \quad L_s = s\,p^2, \quad L_{\lambda_j} = \tfrac12(g_j-1), \quad L_{w_j} = p\,(k_j g_j + \lambda_j g_j^2), \quad L_{k_j} = \ell_j.$$

Because $\log\phi$ splits into independent per-component terms plus the global m and s terms, a mixed second derivative L_{ab} vanishes unless a and b belong to the same component, or are global. The surviving ones are

$$L_{ss} = p^2, \quad L_{w_jw_j} = 2p^2\big(k_jg_j^2 + 2\lambda_j g_j^3\big), \quad L_{\lambda_j w_j} = p\,g_j^2, \quad L_{k_jw_j} = p\,g_j,$$

with all others zero; in particular every $L_{m\,\cdot} = 0$, and $L_{\lambda_j\lambda_j} = L_{k_jk_j} = L_{\lambda_jk_j} = 0$.

We now assemble $H_{ab} = T[(L_{ab}+L_aL_b)\phi]$, grouping the entries by whether they differentiate a degree of freedom.

Entries with no degree-of-freedom derivative (closed form). For any pair that does not differentiate a k, rules R1 and R2 leave only shifted-dof cdfs and their x-derivatives $f = \partial_x F$, $f' = \partial_x^2 F$, …, with no integral surviving:

Global.

$$H_{mm} = f', \qquad H_{ms} = -s\,f'', \qquad H_{ss} = f' + s^2 f'''.$$

Global $\times$ component.

$$\begin{aligned} H_{m\lambda_j} &= -\tfrac12\big(f_{[k_j+2]} - f\big), \quad H_{mw_j} = k_j\,f'_{[k_j+2]} + \lambda_j\,f'_{[k_j+4]}, \\ H_{s\lambda_j} &= \tfrac12 s\big(f'_{[k_j+2]} - f'\big), \quad H_{sw_j} = -s\big(k_j\,f''_{[k_j+2]} + \lambda_j\,f''_{[k_j+4]}\big). \end{aligned}$$

Same component.

$$
\begin{aligned}
H_{\lambda_j\lambda_j} &= \tfrac{1}{4}\big(F_{[k_j+4]} - 2F_{[k_j+2]} + F\big),\\
H_{\lambda_j w_j} &= \tfrac{1}{2}k_j\, f_{[k_j+2]} + \tfrac{1}{2}(\lambda_j - k_j - 2)\, f_{[k_j+4]} - \tfrac{1}{2}\lambda_j\, f_{[k_j+6]},\\
H_{w_j w_j} &= k_j(k_j+2)\, f'_{[k_j+4]} + 2\lambda_j(k_j+2)\, f'_{[k_j+6]} + \lambda_j^2\, f'_{[k_j+8]}.
\end{aligned}
$$

Cross component ($i \neq j$).

$$
\begin{aligned}
H_{\lambda_i\lambda_j} &= \tfrac{1}{4}\big(F_{[k_i+2,k_j+2]} - F_{[k_i+2]} - F_{[k_j+2]} + F\big),\\
H_{\lambda_i w_j} &= -\tfrac{1}{2}\Big[k_j\big(f_{[k_i+2,k_j+2]} - f_{[k_j+2]}\big) + \lambda_j\big(f_{[k_i+2,k_j+4]} - f_{[k_j+4]}\big)\Big],\\
H_{w_i w_j} &= k_i k_j\, f'_{[k_i+2,k_j+2]} + k_i\lambda_j\, f'_{[k_i+2,k_j+4]} + \lambda_i k_j\, f'_{[k_i+4,k_j+2]} + \lambda_i\lambda_j\, f'_{[k_i+4,k_j+4]}.
\end{aligned}
$$

Two identities make convenient sanity checks: the whole m-row is minus the x-derivative of the gradient, $H_{mb} = -\partial_x(\partial_b F)$; and for $b \neq s$, $H_{sb} = s\,\partial_x^2(\partial_b F)$.

Entries that differentiate a degree of freedom (convergent integrals). The remaining entries each carry at least one factor ℓ_j, so they stay as the k-derivative integral of §1.1, now with any extra factors p^a or g^n realized as x-derivatives and degree-of-freedom shifts. Written through the basic integral $\partial_{k_j} F$ and its shifted or x-differentiated versions,

$$
\begin{aligned}
H_{mk_j} &= -\partial_x\,\partial_{k_j} F, & H_{sk_j} &= s\,\partial_x^2\,\partial_{k_j} F,\\
H_{\lambda_j k_j} &= \tfrac{1}{2}\big(\partial_{k_j} F_{[k_j+2]} - \partial_{k_j} F\big), & H_{\lambda_i k_j} &= \tfrac{1}{2}\big(\partial_{k_j} F_{[k_i+2]} - \partial_{k_j} F\big)\ \ (i \neq j),
\end{aligned}
$$

$$
H_{w_j k_j} = -f_{[k_j+2]} - k_j\,\partial_{k_j} f_{[k_j+2]} - \lambda_j\,\partial_{k_j} f_{[k_j+4]}, \qquad H_{w_i k_j} = -k_i\,\partial_{k_j} f_{[k_i+2]} - \lambda_i\,\partial_{k_j} f_{[k_i+4]}\ \ (i \neq j),
$$

$$
H_{k_j k_j} = \partial_{k_j}^2 F, \qquad H_{k_i k_j} = \partial_{k_i}\partial_{k_j} F\ \ (i \neq j).
$$

Here $\partial_{k_j} f_{[\dots]}$ is the k-derivative of a shifted pdf — the same log-weighted integrand with one extra $-\partial_x$ — and $\partial_{k_j} F_{[\dots]}$ is the k-derivative of a shifted cdf. The last two entries carry a squared or product logarithmic weight, ℓ_j^2 or $\ell_i\ell_j$. All of these converge for the reason given in §1.1: a single logarithm, or the product of two, grows slowly enough that the weight $1/t$ controls it.

As in the gradient, the only entries that raise the net power of t are the m and s ones, and they emerge as finite density x-derivatives f', f'', f'''. These are the objects whose robust computation §1.5 takes up.

1.3 Gradient with respect to the boundary parameters

Say $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, and we have its probability content in the quadratic region:

$$
q(\boldsymbol{x}) = \boldsymbol{x}'\mathbf{Q}_2\,\boldsymbol{x} + \boldsymbol{q}_1'\boldsymbol{x} + q_0 > 0.
$$

We ask how this probability content changes as we vary the boundary coefficients $(\mathbf{Q}_2, \boldsymbol{q}_1, q_0)$. Writing F for the generalized chi-square cdf of $q(\boldsymbol{x})$, this contained probability is $P(q(\boldsymbol{x}) > 0) = 1 - F(0)$, so its

gradient is minus that of $F(0)$; we therefore work with the cdf F and negate at the end. As explained above ("Why we keep two separate parameter sets"), we obtain the gradient by differentiating the characteristic function of $q(\boldsymbol{x})$ directly in the coefficients, rather than chaining through the non-differentiable conversion to native parameters.

The characteristic function of a quadratic form. Completing the Gaussian integral for $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ gives

$$\phi(t) = \det\big(\mathbf{I} - 2it\,\boldsymbol{\Sigma}\mathbf{Q}_2\big)^{-1/2}\,\exp\Big(\tfrac{1}{2}\,\boldsymbol{p}'\mathbf{M}^{-1}\boldsymbol{p} - \tfrac{1}{2}\,\boldsymbol{\mu}'\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + it\,q_0\Big),$$

where

$$\mathbf{M}(t) = \boldsymbol{\Sigma}^{-1} - 2it\,\mathbf{Q}_2, \qquad \boldsymbol{p}(t) = \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + it\,\boldsymbol{q}_1, \qquad \tilde{\boldsymbol{\mu}}(t) \equiv \mathbf{M}^{-1}\boldsymbol{p}.$$

The matrix $\mathbf{M}^{-1}$ plays the role of a complex "tilted" covariance and $\tilde{\boldsymbol{\mu}}$ that of a tilted mean. Taking the logarithm,

$$\log\phi = -\tfrac{1}{2}\log\det\big(\mathbf{I} - 2it\,\boldsymbol{\Sigma}\mathbf{Q}_2\big) + \tfrac{1}{2}\,\boldsymbol{p}'\mathbf{M}^{-1}\boldsymbol{p} - \tfrac{1}{2}\,\boldsymbol{\mu}'\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + it\,q_0.$$

Differentiating. By the master formula, for any coefficient $\theta \in \{\mathbf{Q}_2, \boldsymbol{q}_1, q_0\}$ we have $\partial_\theta F = T[(\partial_\theta \log\phi)\,\phi]$, so all that is new is $\partial_\theta \log\phi$. Using the standard matrix identity $d\log\det\mathbf{X} = \operatorname{tr}(\mathbf{X}^{-1}d\mathbf{X})$, the factorization $\mathbf{I} - 2it\boldsymbol{\Sigma}\mathbf{Q}_2 = \boldsymbol{\Sigma}\mathbf{M}$, and the inverse-differential $d(\mathbf{M}^{-1}) = 2it\,\mathbf{M}^{-1}(d\mathbf{Q}_2)\mathbf{M}^{-1}$, each derivative is a single line:

$$\partial_{q_0}\log\phi = it, \qquad \nabla_{\boldsymbol{q}_1}\log\phi = it\,\tilde{\boldsymbol{\mu}}, \qquad \nabla_{\mathbf{Q}_2}\log\phi = it\big(\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}'\big).$$

To see where these come from: q_0 appears only in the term $it\,q_0$, giving the first. For $\boldsymbol{q}_1$, we have $\partial\boldsymbol{p}/\partial\boldsymbol{q}_1 = it\,\mathbf{I}$, so the term $\frac{1}{2}\boldsymbol{p}'\mathbf{M}^{-1}\boldsymbol{p}$ contributes $it\,\mathbf{M}^{-1}\boldsymbol{p} = it\,\tilde{\boldsymbol{\mu}}$. And for $\mathbf{Q}_2$, the log-determinant contributes $it\operatorname{tr}(\mathbf{M}^{-1}d\mathbf{Q}_2)$, while $\frac{1}{2}\boldsymbol{p}'\mathbf{M}^{-1}\boldsymbol{p}$ contributes $it\,\tilde{\boldsymbol{\mu}}'d\mathbf{Q}_2\,\tilde{\boldsymbol{\mu}} = it\operatorname{tr}(\tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}'d\mathbf{Q}_2)$, which together give the symmetric matrix shown.

Substituting into the master formula gives the cdf gradient:

$$\boxed{\partial_{q_0}F = T[(it)\phi] = -f(0), \qquad \nabla_{\boldsymbol{q}_1}F = T\big[(it)\,\tilde{\boldsymbol{\mu}}\,\phi\big], \qquad \nabla_{\mathbf{Q}_2}F = T\big[(it)(\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}')\,\phi\big],}$$

all evaluated at the boundary level 0. The first mirrors $\partial_m F$ of §1.1: since q_0 shifts $q(\boldsymbol{x})$ rigidly (just as the native offset m shifts $\tilde{\chi}$), differentiating the cdf in q_0 is minus differentiating it in its level, and rule R2 gives $\partial_{q_0}F = -f(0)$, the density at the boundary. The other two are density-type Gil-Pelaez integrals, weighted respectively by the vector $\tilde{\boldsymbol{\mu}}$ and the matrix $\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}'$.

The symmetry of the $\mathbf{Q}_2$ derivative. Because $\nabla_{\mathbf{Q}_2}\log\phi$ is symmetric, so is the gradient $\mathbf{G} \equiv \partial F/\partial\mathbf{Q}_2$: it is the symmetric matrix defined by $dF \approx \operatorname{tr}(\mathbf{G}\,d\mathbf{Q}_2)$ for symmetric perturbations $d\mathbf{Q}_2$. This is what a matrix-space optimizer wants, since a step $\mathbf{Q}_2 \leftarrow \mathbf{Q}_2 - \eta\mathbf{G}$ keeps $\mathbf{Q}_2$ symmetric.

Evaluating the gradient efficiently. Each derivative has the form $(it)\cdot(\text{weight})\cdot\phi$, a single Gil-Pelaez integral. The only apparent cost is that $\mathbf{M}^{-1}$ looks like a fresh $d\times d$ complex inverse at every quadrature node. But the conversion to native parameters already supplies the eigendecomposition $\boldsymbol{\Sigma}^{1/2}\mathbf{Q}_2\boldsymbol{\Sigma}^{1/2}=\mathbf{V}\,\mathrm{diag}(w_j)\mathbf{V}'$, whose eigenvalues w_j are exactly the weights. In that basis,

$$\mathbf{M}^{-1}=\boldsymbol{\Sigma}^{1/2}\,\mathbf{V}\;\mathrm{diag}\Big(\tfrac{1}{1-2it\,w_j}\Big)\,\mathbf{V}'\boldsymbol{\Sigma}^{1/2},$$

so the per-node cost collapses to a diagonal scaling by $1/(1-2it\,w_j)$ — the very factor of rule R1. And since $\mathbf{M}^{-1}$ and $\tilde{\boldsymbol{\mu}}$ are shared across all the entries, a single integration returns the whole $(\mathbf{Q}_2,\boldsymbol{q}_1,q_0)$ gradient at once.

The classification error. The optimal quadratic boundary and the error it produces are derived by Das and Geisler,[1] and the gradient of that error follows directly. Writing the error as $\mathcal{E}=p_0\,P(\eta>0\mid 0)+p_1\,P(\eta<0\mid 1)$ for a log-odds boundary η (the same coefficients enter both classes, with opposite sign), its gradient is $\nabla_\theta\mathcal{E}=p_0\,\nabla_\theta P_0+p_1\,\nabla_\theta P_1$, each term being the gradient above, evaluated with that class's $(\boldsymbol{\mu}_y,\boldsymbol{\Sigma}_y)$ and the appropriate sign and tail.

1.4 Hessian with respect to the boundary parameters

For the second derivatives we use the same second-order master formula as in §1.2, now ranging over $a,b\in\{\mathbf{Q}_2,\boldsymbol{q}_1,q_0\}$:

$$\partial_a\partial_b F=T\big[(L_{ab}+L_aL_b)\,\phi\big],\qquad L_a\equiv\partial_a\log\phi,\;\; L_{ab}\equiv\partial_a\partial_b\log\phi.$$

On top of the first derivatives L_a from §1.3, we need only the second derivatives L_{ab} of $\log\phi$. Differentiating the building blocks of §1.3 once more (writing δ for a perturbation),

$$\delta\boldsymbol{p}=it\,\delta\boldsymbol{q}_1,\qquad \delta(\mathbf{M}^{-1})=2it\,\mathbf{M}^{-1}\,\delta\mathbf{Q}_2\,\mathbf{M}^{-1},\qquad \delta\tilde{\boldsymbol{\mu}}=it\,\mathbf{M}^{-1}\delta\boldsymbol{q}_1+2it\,\mathbf{M}^{-1}\,\delta\mathbf{Q}_2\,\tilde{\boldsymbol{\mu}}.$$

Since $L_{q_0}=it$ does not depend on any coefficient, every second derivative involving q_0 vanishes. Differentiating the other two,

$$L_{\boldsymbol{q}_1\boldsymbol{q}_1}=(it)^2\mathbf{M}^{-1},\qquad L_{\boldsymbol{q}_1\mathbf{Q}_2}[\,\cdot\,,\delta\mathbf{Q}_2]=2(it)^2\mathbf{M}^{-1}\delta\mathbf{Q}_2\,\tilde{\boldsymbol{\mu}},$$

$$L_{\mathbf{Q}_2\mathbf{Q}_2}[\,\cdot\,,\delta\mathbf{Q}_2]=2(it)^2\big(\mathbf{M}^{-1}\delta\mathbf{Q}_2\,\mathbf{M}^{-1}+\mathbf{M}^{-1}\delta\mathbf{Q}_2\,\tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}'+\tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}'\delta\mathbf{Q}_2\,\mathbf{M}^{-1}\big).$$

We now assemble the Hessian block by block, $H_{ab}=T[(L_{ab}+L_aL_b)\phi]$.

The q_0 **row** mirrors the m-row of §1.2. Because $L_{q_0,b}=0$, only the product term survives, $H_{q_0,b}=T[(it)L_b\,\phi]$, and by rule R2 the extra factor it is one more q_0-derivative (again because q_0 shifts $q(\boldsymbol{x})$ rigidly). So each entry of this row is the q_0-derivative of the corresponding gradient entry, obtained by inserting one more factor it with no new integral:

$$\boxed{H_{q_0q_0}=f'(0),\qquad H_{q_0,\boldsymbol{q}_1}=\partial_{q_0}\big(\partial_{\boldsymbol{q}_1}F\big),\qquad H_{q_0,\mathbf{Q}_2}=\partial_{q_0}\big(\partial_{\mathbf{Q}_2}F\big).}$$

For the $\boldsymbol{q_1}$–$\boldsymbol{q_1}$ **block**, the two contributions combine neatly: $L_{\boldsymbol{q}_1\boldsymbol{q}_1} + L_{\boldsymbol{q}_1} L'_{\boldsymbol{q}_1} = (it)^2\mathbf{M}^{-1} + (it)^2\tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}' = (it)^2(\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}')$, which is the $\mathbf{Q}_2$-gradient weight times an extra factor it. Hence

$$\boxed{H_{\boldsymbol{q}_1\boldsymbol{q}_1} = T\big[(it)^2(\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}')\,\phi\big] = \partial_{q_0}\big(\partial_{\mathbf{Q}_2} F\big),}$$

a clean identity: the $\boldsymbol{q}_1\boldsymbol{q}_1$ block equals the q_0-derivative of the $\mathbf{Q}_2$ gradient — that is, the $(q_0, \mathbf{Q}_2)$ entry of the row above.

The $\boldsymbol{q}_1$–$\mathbf{Q}_2$ **block** is a vector for each perturbation $\delta\mathbf{Q}_2$:

$$\boxed{H_{\boldsymbol{q}_1,\mathbf{Q}_2}[\delta\mathbf{Q}_2] = T\Big[\Big(2(it)^2\mathbf{M}^{-1}\delta\mathbf{Q}_2\,\tilde{\boldsymbol{\mu}} + (it)^2\,\tilde{\boldsymbol{\mu}}\,\mathrm{tr}\big((\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}')\delta\mathbf{Q}_2\big)\Big)\phi\Big].}$$

And the $\mathbf{Q}_2$–$\mathbf{Q}_2$ **block** is a bilinear form in two perturbations $\delta\mathbf{A}, \delta\mathbf{B}$:

$$\boxed{\begin{aligned} H_{\mathbf{Q}_2\mathbf{Q}_2}[\delta\mathbf{A}, \delta\mathbf{B}] = T\Big[\Big(&2(it)^2\,\mathrm{tr}\big(\delta\mathbf{A}\,(\mathbf{M}^{-1}\delta\mathbf{B}\,\mathbf{M}^{-1} + \mathbf{M}^{-1}\delta\mathbf{B}\,\tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}' + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}'\delta\mathbf{B}\,\mathbf{M}^{-1})\big) \\ &+ (it)^2\,\mathrm{tr}\big((\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}')\delta\mathbf{A}\big)\,\mathrm{tr}\big((\mathbf{M}^{-1} + \tilde{\boldsymbol{\mu}}\tilde{\boldsymbol{\mu}}')\delta\mathbf{B}\big)\Big)\phi\Big]. \end{aligned}}$$

Every weight here is again built from $\mathbf{M}^{-1}$ and $\tilde{\boldsymbol{\mu}}$, so, exactly as in §1.3, a single Gil-Pelaez pass returns the full boundary Hessian, reusing the same eigen-structured $\mathbf{M}^{-1}$ with only a diagonal rescaling per node.

The obstruction at $s = 0$, and its cure. When $s \neq 0$ the normal term's Gaussian damping $e^{-s^2t^2/2}$ makes all these integrals converge quickly, and the direct inversion above is the method of choice. At $s = 0$ — the classification regime, where a full-rank boundary forces the normal term to vanish (§1.5) — it fails: each Hessian block carries one factor it beyond the gradient, so, after the $1/t$ that T contributes, its integrand decays only like $t^{\,1-D/2}$, at best conditionally convergent for $D \leq 4$. Crucially this afflicts *every* weighted block, not just the threshold (q_0) directions, so the scalar series of §1.5 cannot simply be dropped in — the whole matrix- and tensor-valued weight must be reduced.

The cure is to perform the inversion *before* passing to the tail limit, by expanding the weights in the eigenbasis so that each block collapses to a finite combination of shifted-degree-of-freedom density derivatives — the objects §1.5 evaluates robustly at any D. Recall from §1.3 the eigendecomposition of the pencil $\boldsymbol{\Sigma}^{1/2}\mathbf{Q}_2\boldsymbol{\Sigma}^{1/2} = \sum_j w_j \boldsymbol{v}_j\boldsymbol{v}_j'$, whose eigenvalues w_j are the generalized-chi-square weights. Writing $\boldsymbol{u}_j \equiv \boldsymbol{\Sigma}^{1/2}\boldsymbol{v}_j$ for the corresponding directions in the original coordinates and $g_j(t) \equiv (1 - 2it\,w_j)^{-1}$ for the resolvent factor of the j-th mode,

$$\mathbf{M}^{-1} = \sum_j g_j\,\boldsymbol{u}_j\boldsymbol{u}_j', \qquad \tilde{\boldsymbol{\mu}} = \sum_j g_j\,c_j\,\boldsymbol{u}_j, \qquad c_j \equiv \boldsymbol{u}_j'\boldsymbol{p} = \alpha_j + it\,\beta_j,$$

where $\alpha_j \equiv \boldsymbol{u}_j'\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}$ and $\beta_j \equiv \boldsymbol{u}_j'\boldsymbol{q}_1$ are real constants. Every weight in §1.4 is a polynomial in these two objects, hence, once expanded, a sum of terms of the single form $(it)^p\,g_{j_1}\cdots g_{j_r}\,\phi$. Two elementary facts collapse each such term. First, by rule R1 each factor g_j advances the j-th component's degrees of freedom by two, $g_j\phi = \phi_{[k_j+2]}$, so a product advances several components at once (a repeated index compounding). Second, by rule R2 a power $(it)^p$ is p argument-derivatives, $T[(it)^p\psi] = (-\partial_x)^p T[\psi]$. Together they give the master reduction

$$\boxed{T\big[(it)^p\, g_{j_1}\cdots g_{j_r}\,\phi\big] = (-1)^p\, f^{(p-1)}_{[k_{j_1}+2,\,\ldots,\,k_{j_r}+2]}(x_0),}$$

a density derivative of order $p-1$, evaluated at the parameter set with each listed component's degrees of freedom raised by two. Substituting the eigen-expansions into a block weight and collecting powers of it — from the explicit $(it)^2$ and from the linear-in-t factors c_j — thus turns every block into a finite sum of these shifted-dof density derivatives. For the $\boldsymbol{q}_1\boldsymbol{q}_1$ block,

$$H_{\boldsymbol{q}_1\boldsymbol{q}_1} = \sum_j \boldsymbol{u}_j\boldsymbol{u}_j'\, f'_{[k_j+2]} + \sum_{j,l} \boldsymbol{u}_j\boldsymbol{u}_l'\Big(\alpha_j\alpha_l\, f' - (\alpha_j\beta_l+\alpha_l\beta_j)\, f'' + \beta_j\beta_l\, f'''\Big)_{[k_j+2,\,k_l+2]},$$

and the remaining blocks follow by the same substitution, the tensor blocks $H_{\boldsymbol{q}_1\mathbf{Q}_2}$ and $H_{\mathbf{Q}_2\mathbf{Q}_2}$ merely carrying more eigen-indices and, through their extra c_j factors, higher derivative orders — up to $f^{(4)}$ and $f^{(5)}$ respectively. This reduction is *algebra*, not a convergence fix: the worst term of each block is a shifted density whose own inversion integrand still decays like $t^{\,1-D/2}$ — the raised degrees of freedom exactly cancelled by the raised derivative order — so it too must come from the series of §1.5, evaluated through the mixed-sign convolution (a two-class boundary has weights of both signs), not from inversion. This is also the "ω^2 term diverges" of the background note:[3] what reads as a divergence is a density derivative in disguise, recovered exactly here rather than integrated.

1.5 Robust density derivatives when $s = 0$

Both Hessians rest on the same three x-derivatives of the density, f', f'', f'''. In each parametrization one parameter is a pure threshold shift — m natively, q_0 for the boundary — so its second derivative is just the curvature of F at the threshold, which rule R2 delivers as f'; a factor of s^2 raises this by two orders to f''' where it occurs. Concretely, $H_{mm} = f'$ and $H_{ss} = f' + s^2 f'''$ (§1.2), and the entire boundary q_0-row, $H_{q_0q_0} = f'$ together with $H_{q_0,\boldsymbol{q}_1}$ and $H_{q_0,\mathbf{Q}_2}$ (§1.4), are built from these.

Rule R2 obtains $f^{(n)}$ from an x-weighted Gil-Pelaez integral, so its convergence is governed by the tail of the integrand. Writing $u = 2t$ and $D = \sum_j k_j$ for the total degrees of freedom, that integrand behaves like $u^{\,n-D/2}$ as $u \to \infty$. A non-zero normal term contributes a Gaussian damping factor $\exp(-\tfrac{1}{2}s^2t^2)$ that makes every order converge; but at $s = 0$ this damping is absent, and the integral is only *conditionally* convergent once $n \geq D/2 - \tfrac{1}{2}$, and *divergent* once $n \geq D/2$. At $D = 4$, for instance, f' decays only as u^{-1} and converges conditionally; at $D = 2$ it does not decay at all, and the inversion integral breaks down numerically.

The boundary Hessian lives squarely in this regime. A full-rank quadratic boundary has $s = 0$ exactly, since no flat direction remains to carry a linear normal term, and its total degrees of freedom equal the ambient dimension, $D = \operatorname{rank}(\mathbf{Q}_2) = d$; so a classification problem in $d = 2, 3, 4$ sits right on the divergent and borderline cases. Moreover its weights are generically of mixed sign, because a two-class boundary has $\mathbf{Q}_2 = \tfrac{1}{2}(\boldsymbol{\Sigma}_1^{-1} - \boldsymbol{\Sigma}_0^{-1})$. The regime we must therefore handle robustly is $s = 0$, small D, and mixed-sign weights.

The way out is to stop inverting the characteristic function for these derivatives, and to use instead a representation that converges term by term. At $s = 0$, the variable is a pure weighted sum of non-central

chi-squares, and we first split it by the sign of its weights into a positive part and a (negated) negative part, $q = q_+ - q_-$; each part alone is then a weighted sum with weights of a single sign.

When the weights of a generalized chi-square are all of one sign — so that the underlying quadratic form is an ellipsoid — its density admits a convergent series in chi-square densities of increasing degrees of freedom, due to Ruben.[2] Introducing a positive scale β and writing $M = \sum_j k_j$ for the total degrees of freedom, the series expresses the cdf and pdf as mixtures of central chi-squares,

$$F(x) = \sum_{r \ge 0} a_r \, G_{M+2r}(y), \qquad f(x) = \tfrac{1}{\beta} \sum_{r \ge 0} a_r \, g_{M+2r}(y), \qquad y = \frac{x - m}{\beta},$$

where G_ν and g_ν are the central χ^2_ν cdf and density, and the non-negative coefficients a_r (with $\sum_r a_r = 1$) are fixed by Ruben's recursion. The scale β is arbitrary in principle, but a *genuine* mixture — one with non-negative coefficients — is guaranteed only for $0 < \beta \le \min_j |w_j|$, the smallest weight; we take β to be a fixed fraction of $\min_j |w_j|$, placed just inside this bound. This applies to q_+ and to q_- individually, since each is now a same-sign generalized chi-square.

The point of this form is that its x-derivatives are *closed form*, because differentiating a central chi-square density in its argument only shifts its degrees of freedom. With the shift operator S defined by $(\mathrm{S}g)_\nu \equiv g_{\nu-2}$,

$$\frac{d}{dy}\, g_\nu = \tfrac{1}{2}\big(g_{\nu-2} - g_\nu\big) = \tfrac{1}{2}(\mathrm{S} - \mathrm{I})\, g_\nu,$$

so that, iterating,

$$\frac{d^n}{dy^n}\, g_\nu = 2^{-n}(\mathrm{S} - \mathrm{I})^n g_\nu = 2^{-n} \sum_{i=0}^{n} \binom{n}{i} (-1)^{n-i} g_{\nu-2i}.$$

The first identity is the elementary relation $g'_\nu(y) = g_\nu(y)\big[(\nu-2)/(2y) - \tfrac{1}{2}\big]$, rewritten as a shift. Differentiating the series n times in x, and collecting the factor $1/\beta$ that each derivative picks up from $y = (x-m)/\beta$, we obtain

$$\boxed{f^{(n)}(x) = \beta^{-(n+1)} \sum_{r \ge 0} a_r \; 2^{-n} \sum_{i=0}^{n} \binom{n}{i} (-1)^{n-i} \, g_{M+2r-2i}(y), \qquad y = \tfrac{x-m}{\beta}.}$$

This is the same series, with the same coefficients a_r, only shifted down in degrees of freedom by up to $2n$. There is no new series to build and no integral to perform, and it converges for every D. (For the few low-index terms with $M + 2r - 2i \le 0$, one uses the elementary form of $g^{(n)}_\nu$ directly; only the smallest r are affected, and $M \ge 1$.)

When the original weights were already of one sign, this is the whole answer. Otherwise we must assemble the density from the two parts. Since q_+ and q_- are independent and $q = q_+ - q_-$, the density of q is the

cross-correlation of their densities; carrying out the difference-integral in one variable or the other places the argument x on either factor at will,

$$f(x) = \int_0^\infty f_{q_+}(x+v)\, f_{q_-}(v)\, dv = \int_0^\infty f_{q_+}(u)\, f_{q_-}(u-x)\, du.$$

This is an ordinary, non-oscillatory one-dimensional quadrature — it never meets the oscillatory algebraic tail that defeated the Gil-Pelaez integral — and it returns the density itself cleanly. (A residual normal term $s \neq 0$ would add one further convolution, with $\mathcal{N}(0, s^2)$; but this route is used only at $s = 0$, where that factor is absent.)

Its x-derivatives need one more idea, because a naive differentiation under the integral sign fails in exactly the regime we care about. That x may ride on either factor means a derivative of the convolution may be carried by *either* factor too, so we may write $f^{(n)}$ with the derivatives on f_{q_+} or, equally, on f_{q_-}:

$$f^{(n)}(x) = \int_0^\infty f_{q_+}^{(n)}(x+v)\, f_{q_-}(v)\, dv = (-1)^n \int_0^\infty f_{q_+}(x+v)\, f_{q_-}^{(n)}(v)\, dv.$$

The two are equal as identities, but not equally usable. Each is a convolution of two densities — usually a harmless, smoothing operation — and the catch is that a chi-square density can carry a spike that convolving cannot smooth away. A same-sign generalized chi-square lives on $[m, \infty)$, its smallest value being the offset m (reached when every chi-square in it is zero). At that lower edge the density can blow up: for a single degree of freedom it behaves like $(y-m)^{-1/2}$. On its own this spike is mild enough to integrate, but each x-derivative makes it sharper — like $(y-m)^{M/2-1-n}$ after n derivatives, where M is the degrees of freedom of that part — until it is too sharp to integrate at all, once $n \geq M/2$.

This is why it matters which factor carries the derivatives. In the first form the sharpened spike of $f_{q_+}^{(n)}$ sits where $x + v = m$, i.e. at $v = m - x$. If the threshold is below the positive part's floor, $x < m$, that spike lands in the middle of the integration range, where f_{q_-} is nonzero, so the convolution runs straight over it and no longer converges — even though the true derivative $f^{(n)}(x)$ is perfectly finite. (The plain density, $n = 0$, has only the mild edge and convolves without trouble; only its derivatives fail.) This is not an exotic corner. A full-rank two-class boundary has $\mathbf{Q}_2 = \frac{1}{2}(\mathbf{\Sigma}_1^{-1} - \mathbf{\Sigma}_0^{-1})$, generically indefinite with a low-dof positive part, and its floor m ordinarily lies well above the decision threshold $x = 0$; so this is the *typical* boundary, not a rare one, which is why it slipped past a validation that happened to test only $m < x$.

The second form cures it. There the differentiated — hence singular — factor is $f_{q_-}^{(n)}$, whose edge is at $v = 0$; but its companion $f_{q_+}(x+v)$ vanishes for $x + v < m$, i.e. for $v < m - x$, so the integrand is supported on $v \geq m - x > 0$ and $f_{q_-}^{(n)}$ is sampled only in the smooth interior of q_-. The one remaining singularity is the integrable $(v - (m-x))^{M/2-1}$ edge of the *undifferentiated* f_{q_+} — the same mild edge already present in the density itself. When instead $x \geq m$ the first form is the healthy one: its edge $v = m - x \leq 0$ is out of range and $f_{q_+}^{(n)}$ is sampled strictly inside q_+. The prescription is therefore

$$f^{(n)}(x) = \begin{cases} (-1)^n \displaystyle\int_0^\infty f_{q_+}(x+v)\, f_{q_-}^{(n)}(v)\, dv, & x < m, \\ \displaystyle\int_0^\infty f_{q_+}^{(n)}(x+v)\, f_{q_-}(v)\, dv, & x \geq m : \end{cases}$$

we carry the derivatives on whichever part is being sampled away from its own support edge — on q_- below the floor, on q_+ above it. The two branches agree wherever both converge; they part only at $x = m$, the cusp where the two floors coincide and the density's own derivatives are genuinely singular. That point has measure zero and is stepped around in practice — a negligible normal term s restores the damped inversion there, or the value is filled in by continuity.

This series route replaces the density x-derivatives f', f'', f''' only in the problematic regime of $s = 0$ with small D. Everywhere else — whenever $s \neq 0$, or $s = 0$ with D large enough that the x-weighted integrand converges comfortably — the Gil-Pelaez inversion is both accurate and cheaper, and we keep it. In practice a single density-derivative routine chooses between the two methods from (s, D, n), in the same spirit as the method selection the cdf and pdf routines already perform. This routine is implemented (§2.1), and the native Hessian's m, s blocks and their shifted-dof densities (§1.2) call it.

Two things this scalar route does *not* by itself cover, both for the same reason (an object that carries an x-derivative but is not a pure density derivative). First, the Hessian entries that also differentiate a degree of freedom (§1.2, e.g. $H_{w_j k_j}$) involve $\partial_x \partial_{k_j} F$, which Ruben's series does not produce; at $s = 0$ with small D these keep the inversion route. Second, the boundary Hessian (§1.4) is built from the same density x-derivatives but *weighted* by $\mathbf{M}^{-1}$ and $\tilde{\boldsymbol{\mu}}$; those weighted integrals diverge at $s = 0$ just as f', f'' do, and are made robust by the shifted-dof reformulation of §1.4, which expands each block into shifted-dof density derivatives that this routine then evaluates.

2 Code implementations: benchmarks against finite-difference

2.1 Gradient and Hessian with respect to the $\tilde{\chi}$ parameters

`cdf_grad_gx2` was benchmarked against finite differencing of `gx2cdf` on three distributions (same/mixed-sign weights, $s \neq 0/s = 0$; ground truth a near-exact analytic evaluation — `vpa` at $s \neq 0$, tight double precision at $s = 0$ — and FD swept over step size and matched to its own best-achievable error). Python results are from the `gx2-py` port (`cdf_grad_gx2`), run under the identical protocol, same parameters, same cases:

case	$\boldsymbol{w}$	$\boldsymbol{k}$	$\boldsymbol{\lambda}$	s	m	x	P
1	$[1, -2]$	$[2, 3]$	$[1, 0.5]$	1.5	1	2	8
2	$[1, -2, 3]$	$[1, 2, 1]$	$[0.5, 1, 0.5]$	1	2	5	11
3	$[1, -1.5]$	$[1, 1]$	$[0.5, 0.3]$	0	0	0.5	8

Case 3 is the hard corner: $s = 0$, small total dof ($D = 2$), mixed-sign weights.

Python (`cdf_grad_gx2`):

case	quantity	analytic time	FD time	analytic error	FD error	accuracy gain
1 ($s \neq 0$)	gradient	54 ms	49 ms	$6e-17$	$1e-11$	$\sim 2e5$
1 ($s \neq 0$)	Hessian	0.29 s	0.41 s	$7e-14$	$7e-9$	$\sim 1e5$
2 ($s \neq 0$)	gradient	0.11 s	87 ms	$8e-17$	$7e-12$	$\sim 8e4$
2 ($s \neq 0$)	Hessian	0.88 s	1.0 s	$5e-13$	$1e-9$	$\sim 2e3$
3 ($s = 0$)†‡	gradient	42 s	122 s	—	$1.6e-4$	—
3 ($s = 0$)†‡	Hessian	287 s	(not obtained)	—	—	—

MATLAB (`cdf_grad_gx2`):

case	quantity	analytic time	FD time	analytic error	FD error	accuracy gain
1 ($s \neq 0$)	gradient	11 ms	11 ms	$3e-17$	$7e-12$	$\sim 2e5$
1 ($s \neq 0$)	Hessian	61 ms	85 ms	$1e-16$	$2e-8$	$\sim 2e8$
2 ($s \neq 0$)	gradient	17 ms	15 ms	$6e-17$	$6e-12$	$\sim 1e5$
2 ($s \neq 0$)	Hessian	0.15 s	0.16 s	$1e-15$	$1e-8$	$\sim 9e6$
3 ($s = 0$)†‡	gradient	1.1 s	0.52 s	$2e-6$	$9e-6$	~ 5.7
3 ($s = 0$)†‡	Hessian	2.2 s	2.8 s	$1e-4$	$1e-3$	~ 12

†Excludes the degree-of-freedom-derivative entries, unreliable at $s = 0$ small dof for both methods and tracked as open item 1 in §3. ‡Case 3 in Python departs from the matched-accuracy protocol because a single evaluation there already costs seconds; see open item 6 in §3.

2.2 Gradient and Hessian with respect to the boundary coefficients

`cdf_grad_norm_quad` was benchmarked against finite differencing on three boundaries (flat vech($\mathbf{Q}_2$)+$\boldsymbol{q}_1$+q_0 basis; ground truth a tight-tolerance analytic evaluation; FD swept over step and matched to its own best error). Case 2's same-sign $s = 0$ series is deterministic and tolerance-independent, so its accuracy is instead controlled by term count (the `n_ruben` parameter), read at the smallest N whose error already beats FD's. Python results are from the `gx2-py` port (`cdf_grad_norm_quad`), same parameters, same cases:

case	$\boldsymbol{\mu}$	$\boldsymbol{\Sigma}$	$\mathbf{Q}_2$	$\boldsymbol{q}_1$	q_0	s	P
1	$[1, 2, 0.5]$	$\mathrm{diag}(2, 3, 1)$	$\mathrm{diag}(1, -1, 0)$	$[0, 0, 1]$	0	$\neq 0$	10
2	$[1, 2]$	$[[2, 1], [1, 3]]$	$[[1, 0.5], [0.5, 1]]$	$[-1, 0]$	0	0	6
3	$[1, 2]$	$[[2, 1], [1, 3]]$	$[[1, 0.5], [0.5, -1]]$	$[-1, 0]$	0	0	6

Case 1 has a purely linear third dimension, giving a nonzero normal term s (direct-inversion route); case 2 is a full-rank quadratic with a positive-definite $\mathbf{Q}_2$, so $s = 0$ with same-sign weights (shifted-dof route via

the Ruben series); case 3 is a full-rank quadratic with an indefinite $\mathbf{Q}_2$, so $s = 0$ with mixed-sign weights — the generic two-class boundary.

Python (`cdf_grad_norm_quad`):

case	quantity	analytic time	FD time	analytic error	FD error	accuracy gain
1 ($s \neq 0$)	gradient	14 ms	60 ms	$6e-17$	$8e-8$	$\sim 1e9$
1 ($s \neq 0$)	Hessian	24 ms	0.61 s	$6e-15$	$2e-6$	$\sim 3e8$
2 ($s = 0$, same)	gradient	3.5 ms	9.9 ms	$8e-13$	$1e-11$	~ 17
2 ($s = 0$, same)	Hessian	14 ms	61 ms	$3e-8$	$4e-8$	~ 1.4
3 ($s = 0$, mixed)	gradient	35 s	79 s	$2e-8$	$2e-5$	$\sim 1.4e3$
3 ($s = 0$, mixed)	Hessian	102 s	482 s	$9e-10$	$9e-5$	$\sim 1e5$

MATLAB (`cdf_grad_norm_quad`):

case	quantity	analytic time	FD time	analytic error	FD error	accuracy gain
1 ($s \neq 0$)	gradient	6.2 ms	14 ms	$2e-16$	$8e-8$	$\sim 4e8$
1 ($s \neq 0$)	Hessian	20 ms	150 ms	$2e-15$	$1e-6$	$\sim 6e8$
2 ($s = 0$, same)	gradient	11 ms	13 ms	$8e-13$	$1e-11$	~ 14
2 ($s = 0$, same)	Hessian	60 ms	84 ms	$3e-8$	$5e-8$	~ 1.7
3 ($s = 0$, mixed)	gradient	0.53 s	1.2 s	$9e-8$	$5e-5$	~ 491
3 ($s = 0$, mixed)	Hessian	2.5 s	7.7 s	$2e-7$	$5e-3$	$\sim 2e4$

3 Ongoing work and open items

- [ ] **1. Robust analytic k-derivatives at $s = 0$, small dof.** The native Hessian entries that differentiate a degree of freedom against the argument — $H_{w_j k_j}, H_{\lambda_j k_j}, H_{k_i k_j}, H_{s k_j}$, built from $\partial_x \partial_{k_j} F$ and $\partial_{k_i} \partial_{k_j} F$ — remain on the Imhof inversion (`gx2_imhof` `'k_deriv'` / `'kk_deriv'`), because Ruben's series (§1.5) produces density derivatives $f^{(n)}$ but **not** derivatives in the degrees of freedom. They are accurate at $s \neq 0$ (Gaussian damping) or $s = 0$ with adequate dof, and lose accuracy only in the $s = 0$, small-dof corner. A robust fix needs new math — a series representation differentiable in k — which is not derived here. This affects only gx2's own native-parameter Hessian; the boundary-coefficient derivatives (§1.3–§1.4) never differentiate in k, so the quadratic-classification use case is unaffected.
- [ ] **2. Faster mixed-sign density derivatives at $s = 0$.** The same-sign speedup already in place (prefer the Ruben series over the slow inversion) made same-sign $s = 0$ boundary derivatives competitive with finite differences. Mixed-sign weights still take the convolution route in `gx2_dens_deriv` (a cross-correlation of two Ruben series), which is slower; a mixed-sign boundary Hessian at $s = 0$ therefore remains in the seconds. Worth investigating whether the convolution can be accelerated (precomputed series coefficients, a coarser adaptive tolerance, or batching the derivative orders into one quadrature pass) to bring mixed-sign $s = 0$ down to the same-sign level.

- [ ] **3. Density-derivative cusp at the alignment point $x = m$ (mixed sign, $s = 0$).** The mixed-sign convolution of §1.5 computes $f^{(n)}$ by carrying the x-derivatives on whichever split part is sampled in its smooth interior — on q_- when the threshold $x < m$, on q_+ when $x \geq m$, where m is the offset, i.e. the floor of q_+. Each branch works because the companion (undifferentiated) factor vanishes near the differentiated part's support edge and so shields its spike. These shields both fail at one point, $x = m$, where the two floors coincide — q_+ at its floor m and q_- at its floor 0 simultaneously — and the shielding gap $|m - x|$ closes. There the density f_q itself has a genuine cusp: it stays continuous, but its derivatives $f^{(n)}(m)$ diverge at low dof (the cross-correlation of two $(\cdot)^{-1/2}$ edges leaves a kink). So there is no finite value to return at $x = m$, and in a small neighborhood of it either branch samples its singular factor close to the edge and loses accuracy. This is harmless for the classification use case, where the derivatives are wanted at the decision threshold $x_0 = 0$ while m — the boundary's gx2 offset — sits well away from it (it was ≈ 9.5 in the case that first exposed the pre-fix bug); it would matter only for an evaluation within a whisker of $x = m$. A guard is **not yet implemented**: the intended fix is to detect $|x - m| < \epsilon$ and fall back to a damped inversion there (a negligible added normal term s smooths the cusp and makes the Gil-Pelaez integral converge), or to fill the value in by continuity from the two sides. Low priority — it affects no current use, and away from the cusp the two-branch route (§2.2) is exact.
- [ ] **4. Boundary optimization in `IntClassNorm`.** The boundary-coefficient gradient (§1.3) is exactly the sensitivity of a classification probability to its decision boundary: the class probability is $1 - F(0)$ and its gradient in $(\mathbf{Q}_2, \boldsymbol{q}_1, q_0)$ is minus the gradient `cdf_grad_norm_quad` returns. Chained through to the normal's own parameters $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, it gives $\partial(\text{error})/\partial(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ in closed form. This is the primary downstream application of the new machinery: `classify_normals` now exposes it directly as `norm_err_bd_grad` / `norm_err_bd_hess` (or, with custom outcome values, `norm_val_bd_grad` / `norm_val_bd_hess`), gated behind the `bd_deriv` flag. Candidate uses of this output, in roughly decreasing order of how compelling/ready they are:
  - *The optimal boundary as the stationary point of the error.* At the Bayes-optimal quadratic boundary the gradient is exactly zero (confirmed numerically to $\sim 1e-11$–$1e-16$); a Fisher or other hand-chosen boundary instead has a nonzero gradient pointing "downhill" toward lower error. A short example evaluating `norm_err_bd_grad` at both boundaries would make this concrete, and sets up the Hessian below as the curvature of the error landscape exactly at that stationary point.
  - *Excess-risk from a mis-estimated boundary (delta method).* If a boundary is fit from finite data with coefficient covariance $\boldsymbol{\Sigma}_{\text{bd}}$, the expected error above Bayes is second order in the deviation because the gradient vanishes there: $\mathbb{E}[p_e] - p_e^\star \approx \frac{1}{2}\,\text{tr}(\mathbf{H}\,\boldsymbol{\Sigma}_{\text{bd}})$, with $\mathbf{H}$ read off `norm_err_bd_hess`. This turns the empirical normal-approximation checks already in the toolbox (sample-boundary sweeps, `samp_opt_bd` vs. the true boundary) into a closed-form prediction instead of a Monte Carlo sweep, and is the direct match to the excess-classification-error question in Josiah Couch's background note (ref. 3).
  - *Analytic ROC slope.* The classic identity that the ROC slope at a criterion equals the likelihood ratio there follows immediately from the `q0` component of the gradient ($\partial p_h/\partial q_0$, $\partial p_f/\partial q_0$), giving the ROC curve's local slope at any point without sweeping the criterion and re-classifying.
  - *Sensitivity directions of the fitted boundary.* The eigendecomposition of `norm_err_bd_hess` at the optimum separates "stiff" boundary directions, where error rises quickly and the coefficient must be well estimated, from "sloppy" ones the error barely notices — informative when reporting how precisely a fitted boundary (e.g. from vision-experiment cue data) needs to be known.

- *Newton optimization of a constrained boundary.* Restricting the gradient/Hessian to a coefficient subset (e.g. $\mathbf{Q}_2 = \mathbf{0}$, linear-only) turns the search for the best-in-class boundary — such as the Anderson–Bahadur best linear boundary — into a Newton iteration with closed-form derivatives, and lets that boundary's optimality be checked directly (its restricted gradient should vanish).
- *Utility-optimal boundary under custom outcome values.* With `vals` supplied, `norm_val_bd_grad` / `_hess` locates the boundary that maximizes expected payoff rather than minimizes error rate, and quantifies how far the payoff-optimal boundary sits from the error-optimal one.

- [ ] **5. Halley step for `gx2inv`, and quantile density.** The Newton enhancement is already implemented in `gx2inv`; a further option is a Halley step using the second derivative f' (available from the density-derivative route, §2.1) for cubic convergence — likely a small extra speed-up, not yet implemented. The same f' gives the curvature of the quantile function, $d^2x/dp^2 = -f'/f^3$, usable for that Halley step or a smooth quantile interpolant; not otherwise pursued.
- [ ] **6. Python is much slower than MATLAB in the mixed-sign, $s = 0$ corner.** Repeating §2.2's case 3 (mixed-sign, $s = 0$) benchmark in Python:

quantity	MATLAB time	Python time
gradient	1.2 s	~38 s
Hessian	0.17 s	~115 s

Correctness is not in question (agreement to ~ 6 significant digits); the gap is pure runtime. Profiling traces it to `scipy.integrate.quad_vec`'s Gauss-Kronrod stepper calling the integrand once per scalar quadrature node in a Python loop, versus MATLAB's `quadgk`, which batches all nodes of a subinterval into one vectorized call — a genuine API-level difference, not a difference in the math. (An earlier hypothesis — that the Ruben-series convolution itself was the bottleneck — turned out to be a smaller effect; caching its coefficients across quadrature nodes gives a real $\sim 100\times$ speedup on an isolated call but barely moves the full Hessian benchmark, since the dominant cost is the separate Imhof-inversion fallback that many of the Hessian's shifted-dof terms take.)

Attempted fix, reverted. A from-scratch vectorized Gauss-Kronrod-15 integrator (reusing scipy's published node/weight tables, not its internal driver) was written to batch quadrature nodes the way MATLAB does. The vectorized integrand itself is done and validated to machine precision. The batched adaptive-subdivision loop, however, produced `NaN` and failed to converge on the first real test case — likely a catastrophic-cancellation issue in the semi-infinite substitution near the integrand's removable singularity at $u = 0$, though the root cause was not isolated — and was reverted rather than shipped with an open instability. For whoever picks this up next: consider special-casing the $u \to 0$ limit analytically, or reusing scipy's own outer subdivision/error-bookkeeping via a narrow monkey-patch of just its inner per-node loop rather than re-deriving that bookkeeping from scratch; re-profile after any fix rather than assuming a per-node speedup transforms the wall-clock total; and validate broadly (all output modes, both signs, $s = 0$/ $s \neq 0$) before wiring in a replacement, since `imhof` is used throughout the library.

References

1. Das, A., & Geisler, W. S. (2020). *Methods to integrate multinormals and compute classification measures*. arXiv:2012.14331. — Derives the optimal (Bayes) quadratic classification boundary, and the

conversion between the normal-quadratic coefficients $(\mathbf{Q}_2, \boldsymbol{q}_1, q_0, \boldsymbol{\mu}, \boldsymbol{\Sigma})$ and the native $\tilde{\chi}$ parameters $(\boldsymbol{w}, \boldsymbol{k}, \boldsymbol{\lambda}, s, m)$ (implemented in `norm_quad_to_gx2_params`).

2. Das, A. (2025). *New methods to compute the generalized chi-square distribution*. Journal of Statistical Computation and Simulation, 95(12), 2608–2642. doi:10.1080/00949655.2025.2501401. — The gx2 paper: notation, and the Imhof / Gil-Pelaez inversion used throughout.
3. Couch, J. *Background note* (`derivatives/background/To_Abhranil.tex`) — the excess-classification-error motivation and the ω^2 "divergence" resolved by rule R2.